From appointment history to a Booking Agent

Predicting customer returns in a recurring-service business

Edem Asare Agbo | Project article | 13 October 2026

The business problem

A salon builds relationships through repeat visits, but deciding when to contact each customer takes time. Staff may call customers without knowing whether they are due for a visit, or send the same reminder to everyone. Appointment history offers a starting point for more informed follow-up.

I built this project to explore who is likely to return and approximately when. The current system resolves customer identities, removes likely duplicate or rescheduled appointments, compares prediction models, and exports probabilities and estimated return dates. The longer-term idea is a Booking Agent that can handle routine scheduling conversations while staff focus on delivering services.

What the project delivers

Clean histories. Combine name aliases and phone typos under a unique customer identity, then collapse closely spaced reschedules.

Measured predictions. Evaluate models on later appointments and compare probability error against a population baseline.

Customer-level outputs. Estimate upcoming return probabilities conditional on time already elapsed since the last appointment.

Scope of the evidence

The data records appointment requests. A recorded appointment is not proof of attendance. The results reported here come from the owner's local run on private business data. No customer records are included in the repository. The downloadable example uses fictional data and demonstrates the workflow, not business performance.

Messaging, live calendar access, negotiation and autonomous booking are proposed extensions. They are not implemented in this project.

1. Clean the customer identity before modelling

The original name-plus-phone key could split one person into multiple histories. The cleaner applies four explicit rules: the same normalized phone merges aliases; similar names with one or two changed phone digits merge; identical normalized full names merge across different numbers; unmatched records stay separate.

A canonical display name and separate CustomerID identify the merged history. Audit files retain original names, phones and merge reasons. The rules are operational assumptions: shared phones, common names and transitive merges can incorrectly join people, so the mapping should be reviewed.

For each customer, consecutive appointments less than three calendar days apart are treated as duplicates or reschedules. The latest date and time is retained. A gap of exactly three days begins a new cluster. This improves consistency but can remove genuine close-together visits; the rule should be checked against the business process.

2. Predict return windows, then time to return

The logistic-regression baseline asked whether a customer had another appointment within a fixed number of days after a visit. The survival workflow instead models the interval until the next appointment. An observed next appointment is an event. An unfinished interval is right-censored at the date through which records are complete, rather than labelled as someone who will never return.

Historical features include visit count, average and variability of visit gaps, recent visit frequency, and appointment calendar information. Features use history available at the appointment anchor. Cox regression provides an interpretable regularized baseline. A random survival forest allows nonlinear patterns and feature interactions.

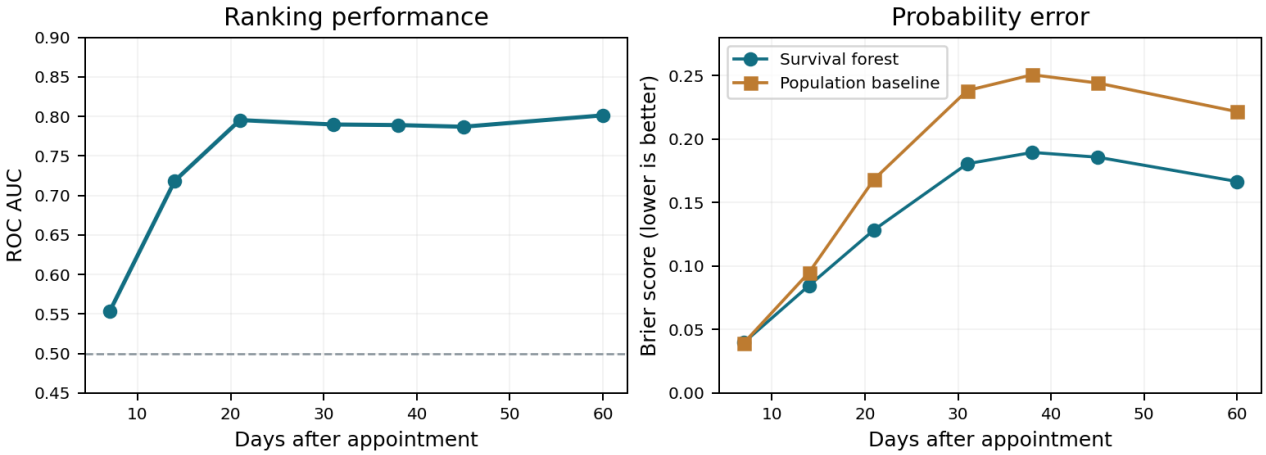
3. Evaluate without future outcomes leaking into training

The workflow uses date boundaries at the 60th and 80th percentiles for validation and test periods. Training intervals are censored at each boundary. Evaluation anchors require 60 days of follow-up within their period. The model with the lowest mean validation Brier score across the seven horizons is selected before the final test is evaluated. It is then refitted on all available history for deployment.

This requires enough history for each period. Repeated observations of the same customer are appropriate for predicting existing customers, but these metrics do not measure performance on entirely unseen customers.

Results from the cleaned business data

The survival forest achieved lower validation Brier error than Cox at every tested horizon. The selected forest was evaluated on 850 later appointment anchors. These are appointment rows, not necessarily 850 distinct customers.



Window	Test AUC	Forest Brier	Baseline Brier
7 days	0.5533	0.0398	0.0395
14 days	0.7185	0.0847	0.0949
21 days	0.7953	0.1286	0.1684
31 days	0.7897	0.1805	0.2381
38 days	0.7890	0.1894	0.2505
45 days	0.7869	0.1856	0.2440
60 days	0.8013	0.1665	0.2215

AUC measures ranking. An AUC of 0.7953 at 21 days means an appointment followed by a return within 21 days tends to score above a non-returning appointment in roughly 79.5% of randomly drawn positive-negative comparisons, with half-credit for ties. It is not 79.5% accuracy or the customer return rate.

Brier measures probability error. It is the mean of (predicted probability - actual outcome) squared. At 21 days, the forest's 0.1286 score is approximately 23.6% lower than the population baseline's 0.1684. At seven days, the forest does not improve probability error over the baseline.

The graph shows reported horizon metrics, not reconstructed ROC curves. The real-data source CSV and uncertainty estimates are not bundled. The scores support further testing at 14-60 days; they do not establish accurate appointment dates or increased bookings from outreach.

Using the prediction in a business

For a customer last seen a days ago, the model computes the chance of returning within the next h days as 1 - S(a+h)/S(a), where S is the estimated probability of no return yet. It also reports conditional P25, median and P75 return dates when supported. These dates are distribution quantiles, not guarantees or confidence intervals.

Predictions must use a complete export through the prediction date. A dataset ending in May cannot establish that a customer has not returned since then. Unsupported horizons remain blank. Before contacting someone, exclude customers with existing bookings, respect messaging permissions and apply a contact cooldown. Return likelihood does not measure whether a message will cause an additional booking; a randomized no-message group is needed to estimate that effect.

The next step: a Booking Agent

A future agent could use return predictions to help choose a follow-up time, draft a personal message with an LLM, interpret customer replies, discuss approved service options, check live calendar availability, and reserve or reschedule an appointment after agreement. Calendar validation and duplicate-booking prevention would protect scheduling consistency. The business goal is less routine administration and more time providing services.

Richer history, completed-visit records, customer preferences and better cleaning may improve the model. First-time and recurring customers can be studied separately. Automatically removing every one-time customer would change the population and could bias the evaluation; any filter must use information available at the prediction date.

Run the fictional example

The repository includes examples/appointments_sample.csv: 3,034 generated rows for 50 fictional customers, with history from 2020 and simulated observation through 28 May 2026. Mock phones start with zero. The fixture includes aliases, phone typos, one-day reschedules and one-time customers. Instructions are in examples/README.md. Outputs use separate demo directories to protect real-data models.

Install requirements-survival.txt, clean the example CSV, train with --observation-end 2026-05-28, and predict from that same date. Synthetic metrics are only a software demonstration and must not be presented as the business results above.

Method references

scikit-survival: Evaluating survival models.
scikit-survival.readthedocs.io/en/stable/user_guide/evaluating-survival-models.html
Source code and runnable workflow: github.com/Ed-project-1/barbershop-ml